Synchronization

Lecture 11

Computer Networks and Distributed Programming

Where we are: the increment that went missing

Threads share the globals, the heap and the descriptors — which is why a chat server can keep the list of its clients in a variable. It is also why `count++` printed 431027 instead of 800000.

- The problem was never the increment: it is that a sequence of operations on shared state can be interrupted halfway, and nothing in C makes it indivisible.
- So we need to say: from here to here, one thread at a time.
- And we need the other half, which the counter did not show: a thread with nothing to do must be able to wait until another one has done something.

Two tools for the two problems — mutex and condition variable — a third that counts, and one structure that puts them together.

The shared state of a server

- The table of connected clients: one thread adds an entry on `accept`, another walks it to relay a line, a third removes an entry when a client leaves.
- The counters you print on a status line: bytes served, clients seen, errors.
- A cache or a table of rooms: read by everybody, written now and then.
- The queue of work waiting to be done, which we build at the end of this lecture.

Each of these has an invariant — `n` is the number of valid entries, every entry below `n` is an open descriptor — and an invariant is exactly what an update that stops halfway breaks.

What a mutex is

Definition

A mutex is a lock with two states, open and locked, held by at most one thread at a time. It lives in a variable of type `pthread_mutex_t`.

- The thread that locks it becomes its owner, and only the owner may unlock it: a mutex is not passed from hand to hand.
- A thread that finds it locked sleeps until it is free — it does not spin, and the kernel gives the processor to somebody else.
- Every thread that touches the data must lock the same mutex. The protection is cooperative: nothing forces it.

The mutex knows nothing about what it protects. That association exists only in the code you write.

The mutex API

```c
#include <pthread.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;   /* a static one: nothing else to do */

int pthread_mutex_init(pthread_mutex_t *m, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *m);   /* nobody may be holding it */

int pthread_mutex_lock(pthread_mutex_t *m);      /* waits until it is free */
int pthread_mutex_trylock(pthread_mutex_t *m);   /* EBUSY instead of waiting */
int pthread_mutex_unlock(pthread_mutex_t *m);
```

- `init` is for a mutex that is not a global: one inside a structure you `malloc`. `attr` `NULL` means the defaults, which is what you want.
- They return an error code, like every pthread call, and not -1 with `errno`.
- `trylock` is not the polite version of `lock`: use it only when there is something else you can do meanwhile.

The counter, fixed

```c
static int count = 0;
static pthread_mutex_t count_mtx = PTHREAD_MUTEX_INITIALIZER;

static void *bump(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&count_mtx);
        count++;                          /* the whole load-add-store, alone */
        pthread_mutex_unlock(&count_mtx);
    }
    return NULL;
}
```

```
$ ./bump ; ./bump ; ./bump
count = 800000       and 800000, and 800000:  the answer no longer depends on timing
```

The eight threads cannot overlap on that line any more — and the program is slower, because now they queue for it. That is the trade.

A mutex protects data, not lines of code

- Every access to the variable takes the same mutex — including the reads: a thread that reads a structure while another is halfway through updating it reads something that was never true.
- Two mutexes over one variable protect nothing at all: two threads take one each and both walk in.
- If the structure is modified in one place, a mutex over that critical section is enough. If it is touched from several, the mutex belongs to the structure and travels with it.
- Say out loud what the mutex protects, and write it next to its declaration. It is the only place that association is recorded.

Nothing checks any of this: `count++` without the lock compiles just as well.

One mutex inside the structure it protects

```c
struct clients {
    int fd[MAX];
    int n;                          /* protected by mtx, together with fd[] */
    pthread_mutex_t mtx;       /* the lock travels with the data */
};

struct clients *clients_new(void) {
    struct clients *c = malloc(sizeof *c);
    if (c == NULL) return NULL;
    c->n = 0;
    pthread_mutex_init(&c->mtx, NULL);    /* malloc'ed: no static initializer */
    return c;
}
```

Whoever receives the pointer receives the lock with it, and there is no way to reach the data without seeing it. One lock per structure, not one lock for the whole server.

A shared object with a reference count

```c
struct session { int fd; int refs; pthread_mutex_t mtx; };

void session_hold(struct session *s) {          /* one more thread using it */
    pthread_mutex_lock(&s->mtx);
    s->refs++;
    pthread_mutex_unlock(&s->mtx);
}

void session_release(struct session *s) {
    pthread_mutex_lock(&s->mtx);
    if (-s->refs > 0) { pthread_mutex_unlock(&s->mtx); return; }
    pthread_mutex_unlock(&s->mtx);              /* last one out: nobody can reach it */
    pthread_mutex_destroy(&s->mtx);
    free(s);
}
```

The count is shared state like any other, so it needs the lock; and the object is freed by whoever brings it to zero, not by the thread that created it.

How long to hold a lock

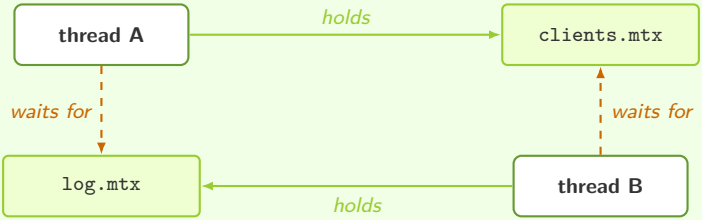
- Lock late and unlock early: whatever sits between the two runs one thread at a time, and that is the part of your server that does not scale.
- Never across a blocking call. A `write` to a slow client can take seconds; if you hold the lock while it happens, every other thread waits for that client. Copy what you need, unlock, then send.
- But do not go to the other extreme: lock, read a value, unlock, then act on it — by then it may be false. What must be indivisible is the decision together with the action.
- Two independent structures get two mutexes, not one: threads working on different things should not queue behind each other.

In one line: hold it for a few instructions on memory, never for I/O.

Two locks, two threads, no progress

A relays a line and needs the log; B writes the log and needs the client table. Both took one lock first.

Neither ever runs again: each holds what the other is waiting for, and a mutex is released only by its owner.



Fix an order on the locks — table, then log — and have every thread take them in that order: the cycle cannot form. Where the order is not known in advance, `trylock`, and on `EBUSY` release everything you hold and start again.

Mutex types, and mutexes across processes

- Default (`PTHREAD_MUTEX_NORMAL`): locking it twice from the same thread deadlocks — the thread waits for itself.
- `PTHREAD_MUTEX_ERRORCHECK` returns `EDEADLK` instead, and refuses an unlock from a thread that does not own it. Worth having while you are debugging.
- `PTHREAD_MUTEX_RECURSIVE` counts, and opens at the matching unlock. Needing one usually means a locked function calls another that locks: the structure is what to fix, not the mutex.

```
1 pthread_mutexattr_t a;
2 pthread_mutexattr_init(&a);
3 pthread_mutexattr_settype(&a, PTHREAD_MUTEX_ERRORCHECK);
4 pthread_mutex_init(&m, &a);
```

A mutex in ordinary memory means nothing to a child after `fork`: the child got a copy. To share one between processes it must live in shared memory, with `PTHREAD_PROCESS_SHARED` set on its attributes.

What a mutex cannot do

A mutex answers one question: *nobody else while I am here*. It does not answer the other one — *wait until there is something to do* — which is what a worker with an empty queue needs.

```
for (;;) {
    pthread_mutex_lock(&m);
    if (queue->count > 0) break;       /* found work: leave the loop still holding m */
    pthread_mutex_unlock(&m);          /* nothing yet: let somebody else in, and ask again */
}
```

- It works, and it burns a processor asking a question whose answer is almost always no.
- Sleep between two tries and you stop burning it, but now you answer late: there is no interval that is right.
- What is missing is a way to sleep until somebody says *the state has changed* — Hoare's condition variables.

Condition variables

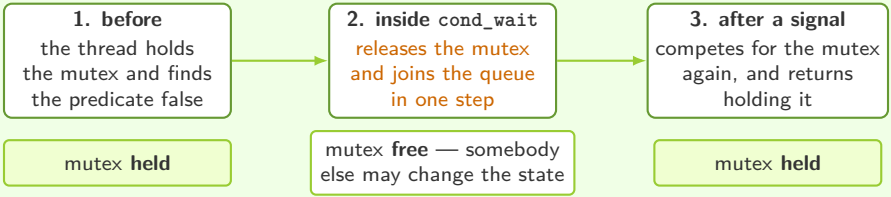
Definition

A condition variable is a queue of threads waiting for something to become true about shared state, with two operations: wait, which puts you in the queue, and signal, which takes somebody out.

- It holds no value and no memory: signalling when nobody is waiting is lost. So the condition itself lives in the shared state — `count > 0` — and never in the variable.
- Which is why it always comes with the mutex that protects that state.

```
1 pthread_cond_t c = PTHREAD_COND_INITIALIZER;      /* or, inside a structure: */
2 int pthread_cond_init(pthread_cond_t *c, const pthread_condattr_t *attr);
3 int pthread_cond_destroy(pthread_cond_t *c);      /* nobody may be waiting on it */
```

What `pthread_cond_wait` does with the mutex



and only now can another thread make the predicate true

- You must hold the mutex when you call it, because you have just read the shared state to find the condition false.
- It releases the mutex and joins the queue as one indivisible step: no signal can slip in between, which is exactly what would be lost if you unlocked yourself first.
- When it returns, the mutex is held again — so the caller is back where it was, and can look at the state once more.

The two sides of the pattern

Waiting for the condition	**Making it true**
`pthread_mutex_lock(&m)`	`pthread_mutex_lock(&m)`
while (condition false)	change the shared state
`  pthread_cond_wait(&c, &m)`	`pthread_cond_signal(&c)`
use the state	`pthread_mutex_unlock(&m)`
`pthread_mutex_unlock(&m)`	

- Both sides hold the mutex: the one that changes the state so that nobody reads it half-changed, the one that waits so that the check and the sleep cannot be separated.
- The right column is three lines and no waiting: making a condition true is never blocking.

Why the wait is inside a `while`

`pthread_cond_wait` has just returned. Is the condition true?

Not necessarily. Which is why the call goes inside a while and never inside an `if`.

- Between the signal and your reacquiring the mutex, another thread may have taken the item: you were told *there was something*, not *there is something for you*.
- A `broadcast` wakes ten threads for one item; nine of them find the condition false again.
- And POSIX allows spurious wakeups: the call may return with nobody having signalled at all.

The `while` costs one extra test and makes all three the same case: look at the state, and if it is not what you need, wait again.

signal or broadcast

- `pthread_cond_signal` wakes exactly one of the waiting threads, chosen non-deterministically. With nobody waiting it does nothing — and nothing is remembered.
- `pthread_cond_broadcast` wakes them all. They still come out one at a time, because each has to reacquire the mutex.
- Use `signal` when any one waiter can handle what changed: one item arrived, one worker is enough.
- Use `broadcast` when the waiters are not interchangeable, or when what changed concerns everybody — the server is shutting down.

`broadcast` where `signal` would do is only slower. `signal` where `broadcast` was needed is a thread that never wakes up.

One producer, two consumers: the consumer

```c
static struct { int a, b; } shared;
static pthread_mutex_t m    = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t  nonempty = PTHREAD_COND_INITIALIZER;
static const struct timespec pause = { 0, 200000000 };   /* 200 ms */

static void *consumer_one(void *arg) {
    for (;;) {
        pthread_mutex_lock(&m);
        while (shared.a == 0)                       /* nothing to take yet */
            pthread_cond_wait(&nonempty, &m);
        printf("consumer 1: a = %d\n", shared.a);
        shared.a--;
        pthread_mutex_unlock(&m);
        nanosleep(&pause, NULL);                    /* outside the lock, on purpose */
    }
}
```

The second consumer is the same loop on the same condition variable, and takes `a` and `b` together: two threads in one queue.

The producer, and what the run looks like

```c
static void *producer(void *arg) {
    for (int j = 0; j < 10; j++) {
        pthread_mutex_lock(&m);
        shared.a++; shared.b++;
        printf("producer: a = %d b = %d\n", shared.a, shared.b);
        pthread_cond_signal(&nonempty);      /* one of the two consumers */
        pthread_mutex_unlock(&m);
        nanosleep(&pause, NULL);
    }
    return NULL;
}
```

```
producer: a = 1 b = 1        consumer 1: a = 1
producer: a = 1 b = 2        consumer 1: a = 1
producer: a = 1 b = 4        consumer 2: a = 1, b = 4
```

`a` never gets past one: the consumers empty it as fast as it fills. Which of the two gets the item is `signal`'s choice, not yours.

Waiting with a deadline

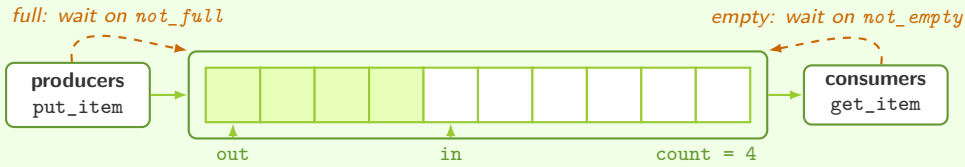
```c
int pthread_cond_timedwait(pthread_cond_t *c, pthread_mutex_t *m,
                           const struct timespec *abstime);   /* an instant, not a delay */

struct timespec deadline;
clock_gettime(CLOCK_REALTIME, &deadline);
deadline.tv_sec += 2;

pthread_mutex_lock(&m);
int rc = 0;
while (!ready && rc == 0)
    rc = pthread_cond_timedwait(&c, &m, &deadline);   /* 0, or ETIMEDOUT */
pthread_mutex_unlock(&m);
```

The deadline is absolute so that going round the loop again does not restart the clock. By default it is measured on `CLOCK_REALTIME`, which jumps when somebody sets the clock: `pthread_condattr_setclock(&attr, CLOCK_MONOTONIC)` is what makes it a real timeout.

The bounded buffer



- `n` slots, some producers, some consumers. Full, and a producer waits; empty, and a consumer waits.
- It needs shared state — the array, `in`, `out`, `count` — one mutex over all of it, and two condition variables, because there are two different things to wait for.

It is the same object that will later sit between an `accept` loop and a fixed set of workers.

The shared structure

```c
#define BUF_SIZE 10

typedef struct {
    int buf[BUF_SIZE];
    int in, out, count;          /* 0 <= count <= BUF_SIZE, in and out modulo BUF_SIZE */
    pthread_mutex_t mtx;
    pthread_cond_t not_full;     /* producers wait here */
    pthread_cond_t not_empty;    /* consumers wait here */
} bbuff_t;

void bbuff_init(bbuff_t *b) {
    b->in = b->out = b->count = 0;
    pthread_mutex_init(&b->mtx, NULL);
    pthread_cond_init(&b->not_full, NULL);
    pthread_cond_init(&b->not_empty, NULL);
}
```

One structure, one lock, two queues — and `bbuff_destroy` undoes the three of them before the `free`.

put_item

```
1  void put_item(bbuff_t *b, int item) {
2      pthread_mutex_lock(&b->mtx);
3
4      while (b->count == BUF_SIZE)                  /* full: wait for room */
5          pthread_cond_wait(&b->not_full, &b->mtx);
6
7      b->buf[b->in] = item;
8      b->in = (b->in + 1) % BUF_SIZE;
9      b->count++;
10
11     pthread_cond_signal(&b->not_empty);         /* there is something now */
12     pthread_mutex_unlock(&b->mtx);
13 }
```

It waits on one condition and signals the other one: that is what makes it the producing half.

get_item

```c
int get_item(bbuff_t *b) {
    pthread_mutex_lock(&b->mtx);

    while (b->count == 0)                          /* empty: wait for an item */
        pthread_cond_wait(&b->not_empty, &b->mtx);

    int item = b->buf[b->out];
    b->out = (b->out + 1) % BUF_SIZE;
    b->count--;

    pthread_cond_signal(&b->not_full);          /* one slot free again */
    pthread_mutex_unlock(&b->mtx);
    return item;
}
```

These two functions are the whole interface: the caller never sees the mutex, and so cannot forget it.

Why two condition variables and not one

One variable for both conditions. The buffer is full, producers are waiting on it, and a consumer takes an item and signals. Who wakes up?

Whichever thread the implementation picks — and it may be another consumer, which finds the buffer empty and goes back to sleep. The wakeup is spent, the producers are still waiting, and there is room.

- Nothing is stored: a signal delivered to a thread that cannot use it is simply lost.
- Either one variable per condition, as we did, or one variable and always `broadcast` — waking everybody every time to find the one thread that can proceed.

Hence the rule: one condition variable for each thing threads wait for.

Starting the threads, and stopping them

```
for (int i = 0; i < N_PROD; i++) pthread_create(&prod[i], NULL, producer, bb);
for (int i = 0; i < N_CONS; i++) pthread_create(&cons[i], NULL, consumer, bb);

for (int i = 0; i < N_PROD; i++)
    pthread_join(prod[i], NULL);        /* nobody will put anything else in */

for (int i = 0; i < N_CONS; i++)
    put_item(bb, KILL_PILL);            /* one poison item for each consumer */

for (int i = 0; i < N_CONS; i++)
    pthread_join(cons[i], NULL);        /* each returns as it reads its own */
```

A consumer asleep in `cond_wait` cannot be told to stop: cancelling it would leave the mutex locked. So you stop it the way it is already listening — through the buffer.

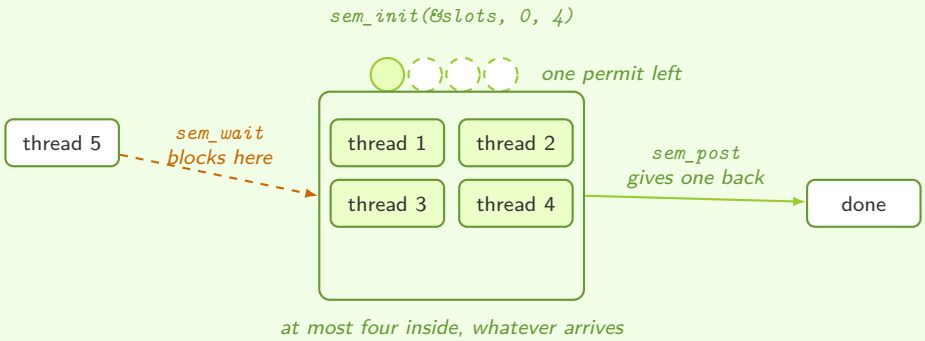
Semaphores

Definition

A semaphore is a non-negative integer with two atomic operations: wait decrements it, blocking while it is zero, and signal increments it. Dijkstra called them `P` and `V`.

- A counting semaphore: the value is the number of permits still available — how many copies of a resource are free.
- A binary semaphore, zero or one, behaves like a mutex with one difference: it has no owner, so any thread may return a permit.
- Which is what makes it also a way to say *it happened*: start at zero, one thread waits, another posts.
- The count is kept for you, so unlike a condition variable there is no predicate of yours to test — and no `while`.

A semaphore is a count of permits



The value is not a flag and not a lock: it is how many may be inside. And since nobody owns a permit, a thread may post one it never took — which is the whole difference from a mutex.

POSIX semaphores

```c
#include <semaphore.h>

sem_t s;
int sem_init(sem_t *s, int pshared, unsigned value);  /* pshared 0: threads of this process */
int sem_wait(sem_t *s);       /* takes one permit; blocks while the value is 0 */
int sem_trywait(sem_t *s);    /* EAGAIN instead of blocking */
int sem_post(sem_t *s);       /* gives one back, and wakes a waiter */
int sem_destroy(sem_t *s);
```

- These are Unix calls, not pthread calls: they return -1 and set `errno`.
- `sem_wait` returns `EINTR` if a signal handler runs while it waits, so it belongs in a loop.
- `sem_post` is the one synchronisation call that may be used inside a signal handler.
- On macOS `sem_init` is deprecated and does not work: use the course container.

The same buffer with semaphores

```
1  sem_t empty, full, lock;          /* initialised to BUF_SIZE, 0 and 1 */
2  void put_item(int item) {
3      sem_wait(&empty);             /* one free slot, or wait for one */
4      sem_wait(&lock);
5      buf[in] = item; in = (in + 1) % BUF_SIZE;
6      sem_post(&lock);
7      sem_post(&full);              /* one more item to take */
8  }
9  int get_item(void) {
10     sem_wait(&full);
11     sem_wait(&lock);
12     int item = buf[out]; out = (out + 1) % BUF_SIZE;
13     sem_post(&lock);
14     sem_post(&empty);
15     return item;
16 }
```

No `while` and no `count`: `empty` and `full` are the count. `lock` is still needed — the two semaphores count items, they do not protect `in` and `out`.

Semaphore or condition variable

- If what you are waiting for is *one unit of something is available*, a semaphore says it in one call and keeps the count for you.
- If it is any other predicate — two items are ready, this client fits in the table, the server is shutting down — then the condition variable is the tool: the predicate is yours and can be anything.
- The order of the two `sem_wait` in `put_item` is not free: take `lock` before `empty` and a full buffer deadlocks. It is the lock ordering rule again.
- And for state that everybody reads and almost nobody writes — the table of connected clients — POSIX has `pthread_rwlock_t`: many readers together, one writer alone.

Three tools, one question each: who may be inside, what am I waiting for, how many at once.

Limiting concurrency with a counting semaphore

```c
static sem_t slots;                          /* how many clients we serve at once */

static void *worker(void *arg) {
    int fd = *(int *) arg; free(arg);
    serve(fd);
    close(fd);
    sem_post(&slots);                        /* the permit goes back */
    return NULL;
}

sem_init(&slots, 0, 64);
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    sem_wait(&slots);                        /* 64 already inside: the loop waits here */
    int *p = malloc(sizeof *p); *p = fd;
    pthread_create(&tid, NULL, worker, p);
}
```

Without it the server creates a thread whatever arrives. With it the number of threads is bounded and the rest wait in the listen queue.

A global that is private to each thread

```
1  static pthread_key_t ctx_key;
2
3  static void ctx_free(void *p) { free(p); }      /* runs when a thread ends */
4
5  pthread_key_create(&ctx_key, ctx_free);         /* once, before the threads */
6
7  pthread_setspecific(ctx_key, ctx);              /* in each worker: my connection */
8  struct ctx *c = pthread_getspecific(ctx_key);   /* anywhere below, without passing it */
9
10 _Thread_local int depth;                         /* C11: same idea, plain variable */
```

- One key, one value per thread, `NULL` everywhere until somebody sets it; the destructor runs once per thread, at its end.
- It is for a context you would otherwise thread through every function — not a way to avoid passing an argument.

Summary

- A mutex answers *one at a time*: every access takes the same one, reads included, held for a few instructions and never across I/O.
- Two locks taken in two orders is a deadlock; one order for the whole program removes it.
- A condition variable is for waiting on a predicate of yours: wait inside a `while`, always holding the mutex, and signal after changing the state.
- `signal` wakes one and `broadcast` all; a signal nobody is waiting for is lost, which is why the condition lives in the shared state.
- The bounded buffer is one mutex and two conditions behind two functions — the queue that will sit between an `accept` loop and its workers.
- A semaphore counts permits: the same buffer without a predicate, and the way to cap how many clients a server handles at once.

Try it

Producers and consumers

Take `n` and `m` from the command line and start `n` producers and `m` consumers over a shared array of 100 integers. Each producer waits a random number of seconds between 1 and 10, then inserts from 1 to 5 random numbers; if it finds the array full, it skips its turn. Each consumer waits until there is something, then prints its thread id and the value it took.

```
$ ./prodcons 3 2        then again with 1 producer and 8 consumers
```

Which of the two waits is a `cond_wait`, and which is not a wait at all? And what goes wrong if your two condition variables become one?